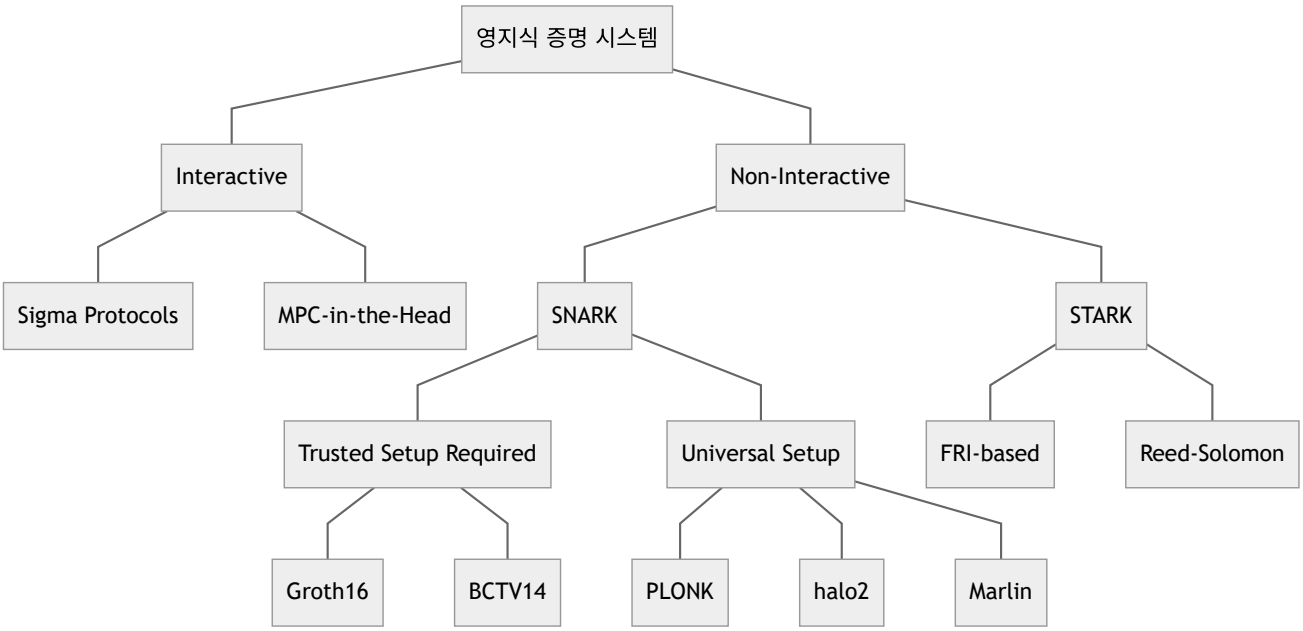
다른 ZK 시스템과의 비교

영지식 증명 시스템 분류

영지식 증명 시스템은 크게 다음과 같이 분류할 수 있습니다:



주요 ZK 시스템 비교

1. Groth16

개요: 2016년 Jens Groth가 제안한 zk-SNARK 시스템

장점

- **매우 작은 증명 크기**: 단 3개의 그룹 원소 (약 192바이트)

- **빠른 검증**: 상수 시간 검증
- **성숙한 생태계**: 광범위한 도구와 라이브러리 지원

단점

- **회로별 신뢰할 수 있는 설정 필요**: 각 회로마다 새로운 설정 의식 필요
- **유연성 부족**: 회로 변경 시 전체 설정 재실행
- **설정 의식의 복잡성**: 신뢰할 수 있는 설정 생성의 어려움

2. STARK vs halo2 비교

| 특징 | STARK | halo2 |

|------|-------|-------|

| **설정** | 투명 (설정 불필요) | 범용 설정 |

| **증명 크기** | 큰 편 (50-200KB) | 작은 편 (수 KB) |

| **검증 시간** | O(log n) | O(1) |

| **양자 저항성** | 있음 | 없음 |

| **구현 복잡도** | 높음 | 중간 |

halo2의 독특한 특징

1. 재귀적 증명 지원

halo2는 네이티브 재귀적 증명을 지원하여 확장성 문제를 해결합니다.

2. Chip 아키텍처

모듈화된 Chip 기반 설계로 코드 재사용성을 높입니다.

3. 유연한 Lookup 시스템

복잡한 표현식과 다중 테이블을 지원하는 고급 Lookup 기능을 제공합니다.

성능 비교

증명 크기 비교

| 시스템 | 증명 크기 | 특징 |

|--------|-----------|------|

| **Groth16** | ~192 bytes | 가장 작음 |

| **halo2** | ~2-10 KB | 작은 편 |

| **PLONK** | ~2-8 KB | halo2와 유사 |

| **STARK** | ~50-200 KB | 큰 편 |

| **Bulletproofs** | ~1-2 KB | 로그 크기 |

halo2의 장단점

장점

1. **범용 설정**: 한 번의 설정으로 모든 회로 지원
2. **재귀적 증명**: 네이티브 재귀적 증명 지원
3. **모듈화**: Chip 기반의 재사용 가능한 설계
4. **유연성**: 복잡한 제약조건과 Lookup 지원
5. **성능**: 실용적인 증명 크기와 검증 시간

단점

1. **신뢰 설정**: 여전히 신뢰할 수 있는 설정 필요

2. **복잡성**: 상대적으로 복잡한 구현

3. **메모리 사용**: 큰 회로에서 높은 메모리 사용량

4. **학습 곡선**: 상당한 학습 시간 필요

선택 가이드

halo2를 선택해야 하는 경우

- ✅ 복잡하고 유연한 회로가 필요한 경우
- ✅ 재귀적 증명이 필요한 경우
- ✅ 모듈화된 설계가 중요한 경우
- ✅ 범용 설정의 이점을 활용하고 싶은 경우

다른 시스템을 고려해야 하는 경우

- ❌ **Groth16**: 매우 작은 증명 크기가 절대적으로 필요한 경우
- ❌ **STARK**: 투명한 설정이 필수인 경우
- ❌ **Bulletproofs**: 범위 증명만 필요한 경우

> **참고**: 각 ZK 시스템은 고유한 장단점을 가지고 있으며, 프로젝트의 요구사항에 따라 적절한 시스템을 선택하는 것이 중요합니다. halo2는 유연성과 성능의 균형을 잘 맞춘 실용적인 선택입니다.